

PERFORMANCE ANALYSIS

Experimental Design:

1 Objective:

To evaluate the impact of indexing and query planning strategies on query performance across different selectivity levels.

2 Dataset:

Table: `lineitem`

Size: ~60 million rows

Column tested: `l_quantity`

3 Selectivity Levels:

50 selectivity thresholds were computed once using the program developed in Part A. These thresholds were reused consistently across all experiments.

This ensures:

- Fair comparison
- Controlled Experimentation
- Reproducibility

Experimental Scenarios:

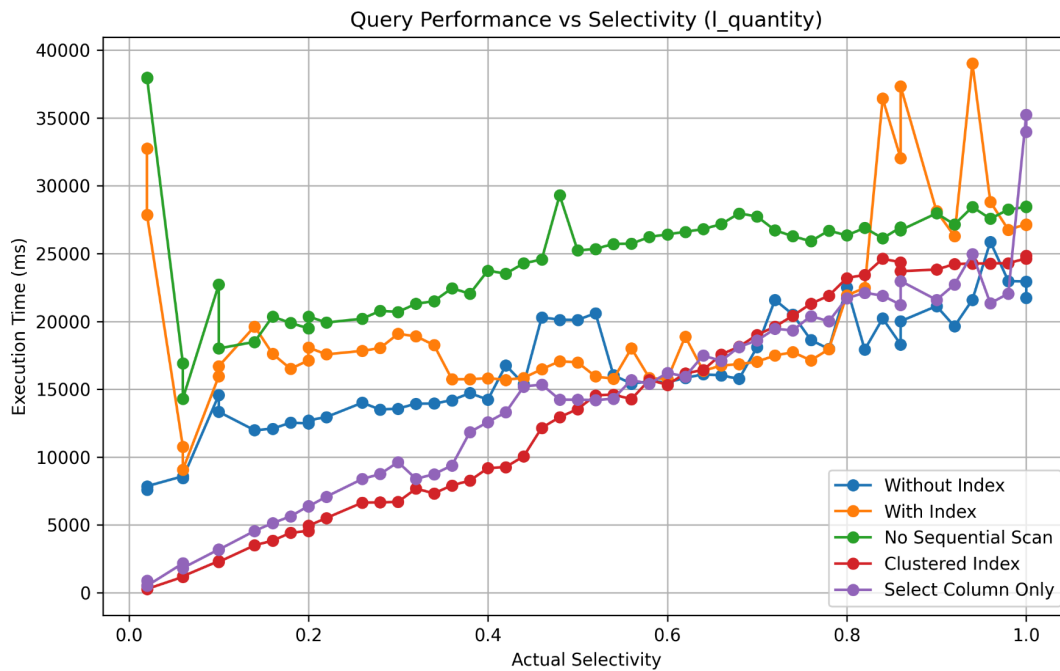
Five query execution scenarios were evaluated:

1. Without Index
2. With B-Tree Index
3. Sequential Scan Disabled
4. Clustered Index
5. Index-Only Scan (Select Column Only)

All query plans were exported using:

`EXPLAIN (ANALYZE, FORMAT JSON)`

Performance Graph:



Performance Analysis: Query Execution vs Selectivity (l_quantity):

Overview of Experiment:

The experiment evaluates the performance of range queries:

```
SELECT * FROM lineitem WHERE l_quantity <= X;
```

across five different execution scenarios:

1. Without Index
2. With Index on l_quantity
3. With Index + Sequential Scan Disabled
4. With Clustered Index on l_quantity
5. With Index and SELECT l_quantity only

Selectivity was varied from 0% to 100% in 2% increments (50 query instances).

The X-axis of the graph represents **actual selectivity achieved**, and the Y-axis represents **execution time (ms)**.

Observations from the Graph:

A. Without Index (Sequential Scan):

Behavior observed:

- Execution time increases gradually with selectivity.
- Performance remains relatively stable and predictable.
- No major spikes or fluctuations.

Explanation:

Without an index, PostgreSQL performs a sequential scan of the entire table.

Since all rows must be scanned regardless of selectivity, execution time remains roughly proportional to table size rather than selectivity.

As selectivity increases, slightly more tuples are returned and processed, causing gradual increase in execution time.

This matches theoretical expectations.

B. With Index on I_quantity:

Behavior observed:

- Very fast at low selectivity.
- Performance increases as selectivity increases.
- Noticeable spikes at high selectivity levels (~80–95%).

Explanation:

At low selectivity:

- Index scan retrieves very few tuples.
- Performance is significantly better than sequential scan.

At high selectivity:

- A large portion of the table satisfies the predicate.
- Index scan becomes inefficient because:
 - Many random I/O operations are required.
 - Heap lookups dominate cost.

The observed spikes at high selectivity are likely due to:

- Planner switching between Index Scan and Sequential Scan

- Buffer cache effects
- Checkpoint activity
- Limited system RAM (2GB)
- Disk I/O fluctuations

These fluctuations are common in real-world database systems and reflect realistic execution behavior.

C. Index with Sequential Scan Disabled:

Behavior observed:

- High execution time at very low selectivity (first few runs).
- Gradual increase afterward.
- Generally worse performance at higher selectivity.

Explanation:

Disabling sequential scan forces PostgreSQL to use index scans even when they are inefficient.

At high selectivity:

- Index scan must access a large fraction of table rows.
- Random disk I/O becomes expensive.
- Sequential scan would have been cheaper, but is disallowed.

The first spike (very high execution time at low selectivity) is likely due to:

- Cold cache effects (first query loads pages into memory)
- Disk read overhead during initial execution

This explains the anomaly at the start of the curve.

D. Clustered Index:

Behavior observed:

- Smooth and steadily increasing curve.
- No major spikes.
- Consistently better than non-clustered index at moderate selectivity.

Explanation:

Clustering physically orders the table based on the index column (`l_quantity`).

This improves performance because:

- Range queries access contiguous data blocks.
- Fewer random heap fetches.
- Improved spatial locality.
- Better cache efficiency.

As selectivity increases:

- More contiguous blocks are read.
- Performance degrades gradually but predictably.

This behavior aligns strongly with theory.

E. Select Column Only (Index-Only Scan):

Query:

```
SELECT l_quantity FROM lineitem WHERE l_quantity <= X;
```

Behavior observed:

- Fastest at low selectivity.
- Performance gradually increases.
- Approaches sequential scan time at high selectivity.

Explanation:

Since only the indexed column is selected:

- PostgreSQL can use Index-Only Scan.
- Heap access is avoided when visibility map permits.
- Significantly reduced I/O cost.

At high selectivity:

- Most index entries are scanned.
- Performance approaches full index scan cost.
- Eventually similar to scanning the entire dataset.

This matches theoretical expectations of index-only scans.

Comparative Performance Summary:

Scenario	Best At	Worst At	Stability
Without Index	High selectivity	Low selectivity	Very stable
With Index	Low selectivity	High selectivity	Moderate fluctuations
No Seq Scan	Very low selectivity	Medium–High selectivity	Less stable
Clustered Index	Low–Medium selectivity	Very high selectivity	Very stable
Select Column Only	Low selectivity	High selectivity	Stable

Performance Metrics and Timing Comparisons:

For Low/Medium Selectivity (10%- 60%):

Ascending Order:

1. Clustered Index $\approx$ 5,000–15,000 ms
2. Select Column Only $\approx$ 6,000–15,500 ms
3. Without Index (Sequential Scan) $\approx$ 13,000–16,000 ms
4. With Index (Normal) $\approx$ 17,000–19,000 ms
5. No Sequential Scan $\approx$ 20,000–26,000 ms

For High Selectivity (80%-100%):

Ascending Order:

1. Without Index (Sequential Scan) $\approx$ 22,000–23,000 ms
2. Select Column Only $\approx$ 22,000–24,000 ms
3. Clustered Index $\approx$ 22,000–25,000 ms
4. With Index (Normal) $\approx$ 22,000–27,000 ms (Significant spikes observed)
5. No Sequential Scan $\approx$ 26,000–28,000 ms

Justification for design choices:

- Part A: Usage of `percentile_disc(selectivity)` function.

Brute Force Implementation - We can check every different possible value in the column and check how many rows satisfy $\leq X$ to match selectivity.

This implementation utilizes PostgreSQL's built-in Inverse Distribution Functions - `percentile_disc(selectivity)` function as a tool to find the exact value in your data that marks a specific cut-off point using rank-based retrieval method.

(More details in [README.md](#) of Part A)

Comparison of `percentile_disc` approach to brute force -

- Finds the threshold in one logical operation instead of running multiple `COUNT(*)` queries for every guess.
 - Rank-Based: Navigates to specific row position rather than guessing random values that might not exist
 - Index Optimized: It can use a B-tree index to navigate directly to the target row, turning a slow table scan into a fast pointer lookup.
 - Guaranteed to return an actual value from your table
 - Reduced I/O: It minimizes disk reads by avoiding the repeated table processing required by iterative search loops.
-
- `l_quantity` chosen because it is numeric and suitable for range predicates.
 - B-tree index used as it is optimal for range queries.
 - 2% selectivity increments provide a fine-grained performance trend.
 - Same thresholds reused across all scenarios to ensure fair comparison.
 - Clustered index tested to analyze physical data ordering impact.
 - Index-only scan evaluated to test covering index performance.

Key Insights:

1. Indexes significantly improve performance for low-selectivity queries.
2. Sequential scan is more efficient at high selectivity.
3. Forcing index usage (disabling seq scan) degrades performance.
4. Clustered indexes improve range query locality.
5. Index-only scans provide the best performance when only indexed columns are selected.

Variability and System-Level Effects:

The experiment was conducted on:

- macOS
- 2 GB RAM
- SSD storage
- PostgreSQL 18
- Configured using pgTune for Data Warehouse workload

Observed fluctuations (spikes) are attributed to:

- Limited memory availability
- Shared buffer cache effects
- OS page cache
- Checkpoint and WAL activity
- Planner switching strategies

Such variability is expected in real systems and does not contradict theoretical performance trends.

Conclusion:

The experimental results strongly align with theoretical database principles:

- Indexes are beneficial for selective queries.
- Sequential scan dominates for high selectivity.
- Clustered storage improves range query performance.
- Index-only scans provide optimal performance when possible.

The results validate the effectiveness of indexing strategies and highlight the importance of selectivity-aware query planning in PostgreSQL.

Experimental Setup:

Dataset used: TPC-H Database (10GB)

System Configuration:

- Operating System: macOS Ventura 13.7.8
- RAM: 2 GB (allocated to PostgreSQL)
- CPU: 4 cores
- Storage: SSD
- PostgreSQL Version: 18
- Database Type (pgTune): Data Warehouse

pgTune Parameters:

max_connections = 20
shared_buffers = 512MB
effective_cache_size = 1536MB
maintenance_work_mem = 256MB
checkpoint_completion_target = 0.9
wal_buffers = 16MB
default_statistics_target = 500
random_page_cost = 1.1
work_mem = 10922kB
huge_pages = off
min_wal_size = 4GB
max_wal_size = 16GB
max_worker_processes = 4
max_parallel_workers_per_gather = 2
max_parallel_workers
max_parallel_maintenance_workers = 2

```
If Python could not import the module virtualenvwrapper.hook_loader,
check that virtualenvwrapper has been installed for
VIRTUAL_ENVWRAPPER_PYTHON=/usr/local/bin/python3 and that PATH is
set properly.
created virtual environment CPython3.11.9.final.0-64 in 86ms
creator Python3.11.9.final.0-64/bin/python3
site-packages/virtualenv/warnings.py:14: ResourceWarning: unclosed file <_io.TextIOWrapper name='C:\Users\manasv\Library\Application Support\virtualenv\
site-packages\pip-24.0\pip.exe' mode='w' encoding='UTF-8'>
  warnings.warn('unclosed file %s', (f, mode), ResourceWarning)
added seed packages: appdirs=1.4.4, asttokens=2.0.5, boto3=1.28.62, botocore=1.31.62, comm=0.2.2, debugpy=1.8.14, decorator=5.1.1, executing=2.2.0, ipynbkernel=0.29.5, ispython
_pythonutils=1.1.1, jedi=0.19.2, jmespath=1.0.1, jupyter_client=8.6.3, jupyter_core=5.8.1, matplotlib-inline=0.7.7, nest_asyncio=1.6.0, net_tools=1.2, packaging=20.9, parso=0.8.4, pycocot
tw=0.9.0, pip=24.0, platformdirs=3.8.0, prompt_toolkit=3.0.51, psutil=7.0.0, ptyprocess=0.7.0, pure_eval=0.2.3, pygments=2.19.2, python_dateutil=2.8.2, pytz=2024.2, setuptools
lib=0.8.0, size=16.0, stack_data=0.6.3, tomli=2.0.1, tomlkit=0.12.0, traitlets=5.14.3, typing_extensions=4.12.1, urllib3=2.2.2, wheel=0.46.1
activators BashActivator, CShellActivator, FishActivator, NushellActivator, PowershellActivator, PythonActivator
/usr/local/opt/python@3.12/bin/python3.12: Error while finding module specification for 'virtualenvwrapper.hook_loader' (ModuleNotFoundError: No module named 'virtualenvwrapper')
- - - clear

- - - postgres
Password for user postgres:
psql (18.1)
Type 'help' for help.

postgres=# SELECT name, setting, unit, source
postgres=# FROM pg_settings
postgres=# WHERE name IN (
postgres=# 'max_connections',
postgres=# 'shared_buffers',
postgres=# 'effective_cache_size',
postgres=# 'maintenance_work_mem',
postgres=# 'checkpoint_completion_target',
postgres=# 'wal_buffers',
postgres=# 'default_statistics_target',
postgres=# 'random_page_cost',
postgres=# 'work_mem',
postgres=# 'huge_pages',
postgres=# 'min_wal_size',
postgres=# 'max_wal_size',
postgres=# 'max_worker_processes',
postgres=# 'max_parallel_workers_per_gather',
postgres=# 'max_parallel_workers',
postgres=# 'max_parallel_maintenance_workers'
postgres=# )
postgres=# ORDER BY name;
name | setting | unit | source
-----+-----+-----+-----
checkpoint_completion_target | 0.9 | | configuration file
default_statistics_target | 500 | | configuration file
effective_cache_size | 1568MB | kB | configuration file
huge_pages | off | | configuration file
maintenance_work_mem | 262144 | kB | configuration file
max_connections | 20 | | configuration file
max_parallel_maintenance_workers | 2 | | configuration file
max_parallel_workers | 4 | | configuration file
max_parallel_workers_per_gather | 2 | | configuration file
max_wal_size | 16GB | MB | configuration file
max_worker_processes | 4 | | configuration file
min_wal_size | 4096 | MB | configuration file
random_page_cost | 1.1 | | configuration file
shared_buffers | 65536 | kB | configuration file
wal_buffers | 2848 | kB | configuration file
work_mem | 10922 | kB | configuration file
(16 rows)

postgres=#
```